

PROJECT PROPOSAL – AWS-Powered Bitrate Drop Visualizer

[\[QUICK VISUALIZATION\]](#)

Objective:

Build a basic AWS-based tool that automatically generates short video clips of known bitrate or audio drops in a 24-hour broadcast file. This will serve as both a personal learning project and a lightweight second method to visualize issues already captured by our internal tools.

Why This Could Help

While Media Proxy is already a strong tool, letting us jump directly to known time stamps, this project would generate short video clips for each anomaly automatically, stored in a structured archive. It gives us a fast visual reference without needing to reopen Media Proxy for each review, and it also creates a long-term visual record of past issues.

While it may overlap with current workflows, it serves as:

- A hands-on learning project using AWS, scripting, and FFmpeg
- A fast-access alternate way to review bitrate/audio anomalies
- A centralized and timestamped archive of validation clips, organized by day and channel
- A historical record of issues faced, useful for retrospectives or recurring problem tracking

How It Works:

- Manually export a 24-hour MP4 stream from Media Proxy
- Upload that file to a structured AWS S3 bucket
- Use a simple AWS-based Python + FFmpeg tool to:
 - Read a CSV list of known issue timestamps
 - Clip short video segments (10 seconds) around each timestamp
 - Upload those clips to S3, organized into folders by channel and date
 - Generate a single HTML summary page per day/channel that lists each clip and links to it (example: “08:23:10 — Bitrate Drop — [Watch Clip]”)

Folder Structure in AWS:

```
s3://bitrate-validation-clips/  
└─ showtime-west/  
    └─ 2025-07-15/  
        ├── full-day-stream.mp4  
        ├── clip-08_23_10-bitrate.mp4  
        ├── clip-13_57_44-audio.mp4  
        ├── report.csv  
        └─ index.html    ← one per day, links to all clips
```

Deliverables:

- S3 storage folders containing:
 - Raw stream
 - Anomaly-based clips
 - Daily HTML summaries of all the clips+their timestamps for when bitrate dropped

AWS Tools necessary:

- S3 bucket
- AWS Lambda – for FFmpeg scripting or EC2 since lambda has stricter limits
- AWS CloudWatch Events – Optional) Schedule or trigger workflows
- AWS Amplify or API Gateway with Lambda – Potential Dashboard building after.

Cost:

None — the project should run entirely on the AWS Free Tier using lightweight compute and storage

Timeframe:

1–2 weeks to build, test, and deliver working model.

Questions:

1. Does this seem viable – not too ambitious?
2. Would you happen to know if Media Proxy offers an API; is there a way we could communicate programmatically w media proxy to request video clips for a given

channel and timestamp using code? Totally fine if not, but if it's available, could eventually automate even the 24 hour clip retrieval part of this workflow.

Possible Workarounds for the No-MediaProxy-API-hurdle:

Media Proxy – No API available for direct bitrate extraction

LogServer [3rd party tool] can analyze MPEG-TS in and generate bitrate graphs – but not great option. Requires manual upload of media proxy files. Defeats purpose kind of.

Possible Courses of Action for Media Sentry-API:

1. Enable API Access - Use the official Telestream Sentry API to programmatically retrieve bitrate + timestamped events – BUT – **NEED a Proper license tier that includes API access**

2. Contact Telestream -

- Describe what we want (e.g., access to bitrate and event logs via API for AWS integration),
- They'll email or grant access to their API documentation (PDF, portal, or internal wiki)

- May have to check or upgrade your **license** or **enable the API module** in your Media Sentry system
- Manually run test requests to confirm which endpoints return bitrate/timestamps, what authentication method is used, and how the JSON or XML is structured

Thennnn It Becomes Scriptable:

- Once we:
- Know the endpoints
- Have valid credentials
- Understand the JSON/XML response format
- You can then **fully automate** it using Python, AWS Lambda, or another scripting tool.

COMPANY WIFI ISSUE:

In making the lambda workspace:

Theres an option to create a VPC for internal **databases and internal services**

▼ **Additional configurations**

Use additional configurations to set up code signing, function URL, tags, and Amazon VPC access for your function.

Networking

Function URL | [Info](#)

Use function URLs to assign HTTP(S) endpoints to your Lambda function.

☐ Enable

VPC | [Info](#)

Connect your function to a VPC to access private resources during invocation.

☐ Enable

Security & governance

Code signing | [Info](#)

Use code signing configurations to ensure that the code has been signed by an approved source and has not been altered since signing.

☐ Enable

Encryption with an AWS KMS customer managed key | [Info](#)

By default, Lambda encrypts the .zip file archive using an AWS owned key.

☐ Enable

Tags | [Info](#)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources, track your AWS costs, and enforce attribute-based access control.

☐ Enable

[Info](#) [Tutorials](#) >

VPC

All Lambda functions run securely inside a default system-managed virtual private cloud (VPC). However, you can also configure your Lambda function to access resources in a custom VPC. A custom VPC defines a private network of resources, such as databases, cache instances, or internal services.

If your AWS Identity and Access Management (IAM) permissions allow you only to create functions that connect to your VPC, you must configure the VPC details when you create the function. If your IAM permissions allow you to create functions that aren't connected to your VPC, you can add the VPC configuration after you create the function.

Was this content helpful?

OR – Could run Media Proxy and Sentry on an EC2 instance – So can access from any laptop?